

API SECURITY & PENETRATION TESTING

Study Notes: HTTP/API Fundamentals, Reconnaissance Methodology & the Transition to Structured Vulnerability Testing

Prepared for: Personal study — Penetration Testing / API Security / Bug Bounty track

Level: Beginner → Intermediate (Reconnaissance Phase)

Companion labs: OWASP Juice Shop, crAPI, Burp Suite Community/Pro

■ How to use these notes

Each section follows: Concept → How It Works → Attacker Mindset → Defensive View → Practical Lab → Tools → Prevention. Work through the labs in a legal environment only (your own lab, Juice Shop, crAPI, or an authorized bug-bounty scope).

Table of Contents

1. How an API Conversation Works
2. Anatomy of an HTTP Request
3. HTTP Methods Explained (GET, POST, PUT, PATCH, DELETE)
4. Understanding HTTP Headers
5. Request Body & User Input
6. Anatomy of an HTTP Response
7. HTTP Status Codes — A Pentester's View
8. Using Burp Suite to Observe Traffic
9. Real API Analysis Workflow
10. Practical Lab — Building an API Inventory (Juice Shop)
11. Mentor Challenge #1 — Privilege Escalation via PATCH
12. API Discovery & Endpoint Mapping Methodology
13. Categorizing Endpoints & Authentication Boundaries
14. Identifying User-Controlled Input
15. JavaScript Recon for Hidden Endpoints
16. Building a Recon Table & Prioritizing Targets
17. The Full Reconnaissance Workflow (Diagram)
18. Mentor Challenge #2 — Prioritizing a Target List
19. From Recon to Vulnerability Testing

- 20. Building Hypotheses (Observation → Hypothesis → Verification)
- 21. Documentation Standards for Findings
- 22. Complete Professional Pentest Workflow
- 23. Mentor Challenge #3 — Final Recon Exercise
- 24. OWASP API Security Top 10 — Quick Reference
- 25. Suggested Learning Roadmap

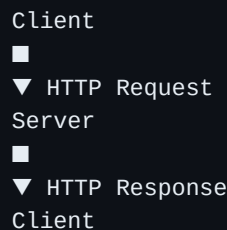
1. How an API Conversation Works

Every API interaction is just a structured conversation between a **client** (browser, mobile app, or tool like Burp/Postman) and a **server**. Understanding this conversation is the foundation of all API testing.

Simple Analogy

Ordering food at a restaurant:

- **You (client):** "Give me a burger." → this is the **Request**
- **Restaurant (server):** "Here is your burger." → this is the **Response**



```
graph TD
    Client -- HTTP Request --> Server
    Server -- HTTP Response --> Client
```

The diagram illustrates the flow of an API conversation. It shows a 'Client' sending an 'HTTP Request' to a 'Server', and the 'Server' sending an 'HTTP Response' back to the 'Client'. The flow is represented by arrows connecting the client and server components.

Attacker Mindset

- A pentester never treats an app as a black box — every click generates a request/response pair worth reading.
- The goal at this stage is **observation**, not exploitation: log every conversation before forming opinions.

Defensive View

- Developers should assume every request is visible and can be modified by a hostile client — never trust client-side logic alone.
- Server-side validation and authorization checks must exist for every request, regardless of what the UI allows.

2. Anatomy of an HTTP Request

```
GET /api/profile HTTP/1.1
Host: api.company.com
Authorization: Bearer eyJ...
User-Agent: Mozilla/5.0
Accept: application/json
```

Breaking Down Line 1

Component	Value	Meaning
HTTP Method	GET	The action being requested
Endpoint	/api/profile	The resource being targeted
Protocol Version	HTTP/1.1	Version of HTTP in use

■ **Real-World Usage**

In Burp Suite's Proxy → HTTP History tab, this exact structure is shown for every captured request. Learning to read it at a glance lets you spot authentication headers, hidden parameters, and endpoint naming patterns within seconds.

3. HTTP Methods Explained

Think of an API as a database with a front door. Each HTTP method performs a different action on a resource.

GET — Read Data

Never modifies data. Used to retrieve resources.

```
GET /users

Server → "Here are all users."
```

POST — Create Something

Used to create a new resource: account, order, comment, etc.

```
POST /users
```

PUT — Replace the Entire Resource

Replaces **all** fields of a resource, even ones not supplied (they may be reset to null/default).

```
PUT /users/15
{
  "name": "Ali",
  "email": "ali@example.com",
  "phone": "0300xxxxxxx"
}
```

PATCH — Update Part of a Resource

Updates only the specified field(s); everything else remains unchanged. Modern REST APIs prefer PATCH for partial updates.

```
PATCH /users/15
{
  "phone": "0300xxxxxxx"
}
```

DELETE — Remove Data

```
DELETE /users/15 → Deletes User 15 (if authorized)
```

Attacker Mindset

- For every endpoint discovered, always test: *"What methods does it allow?"* — an endpoint that only shows GET in the UI may still accept PUT/PATCH/DELETE on the server.

- Use Burp's **OPTIONS** requests or method-tampering to check for undocumented methods (verb tampering).
- PATCH endpoints are high-value: they often accept fields the developer forgot to restrict (e.g. role, isAdmin, balance) — this is Mass Assignment / Excessive Data Exposure (OWASP API3/API6).

Defensive View

- Explicitly allow-list which fields each PATCH/PUT endpoint accepts; never bind request JSON directly to an internal model object.
- Disable or return 405 (Method Not Allowed) on any HTTP method not required by the endpoint's business logic.

■ PRACTICAL LAB — Enumerating Allowed Methods

Field	Detail
Objective	Discover which HTTP methods a given endpoint actually accepts
Tools	Burp Suite (Repeater), curl
Command	<code>curl -i -X OPTIONS https://target/api/users/15</code>
Expected Output	Allow: GET, POST, PUT, PATCH, DELETE header revealing accepted verbs

```
# Manually test each verb with Burp Repeater or curl
curl -i -X PUT https://target/api/users/15 -d '{"role":"admin"}'
curl -i -X PATCH https://target/api/users/15 -d '{"role":"admin"}'
curl -i -X DELETE https://target/api/users/15
```

4. Understanding HTTP Headers

Headers are metadata — think of them as labels on a package that tell the server how to handle the request.

Header	Example	Purpose
Host	api.company.com	Which server should receive this request
Authorization	Bearer eyJ...	Identifies who the client claims to be
Content-Type	application/json	Format of the request body
User-Agent	Mozilla/5.0 ...	Describes the client software
Accept	application/json	Format the client wants back

Attacker Mindset

- The **Authorization** header (JWT/Bearer tokens) is one of the highest-value targets — decode JWTs (jwt.io / [jwt_tool](#)) to inspect claims like `role`, `user_id`, and `exp`.
- Some applications change behavior based on **User-Agent** — worth fuzzing to detect hidden mobile-only or admin-only code paths.
- Missing or inconsistent **Content-Type** handling can sometimes bypass input filters (e.g. sending XML instead of JSON to reach a different parser).

Defensive View

- Never trust identity information taken from headers you can't cryptographically verify.
- Validate JWT signatures server-side on every request; do not just decode and trust the payload.

5. Request Body & User Input

```
{  
  "email": "ali@example.com",  
  "password": "Password123!"  
}
```

The request body is where user-controlled input usually lives, making it the primary surface for later vulnerability testing (SQL Injection, NoSQL Injection, Mass Assignment, etc.).

Attacker Mindset

- Every JSON key in the body is a candidate parameter to fuzz, not just the ones shown in the UI form.
- Try adding *extra* keys that were never in the UI (e.g. "isAdmin": true) to test for Mass Assignment.

6. Anatomy of an HTTP Response

```
HTTP/1.1 200 OK
Content-Type: application/json

{
  "token": "eyJ..."
}
```

Every response has three parts: **Status Code**, **Headers**, and **Body**.

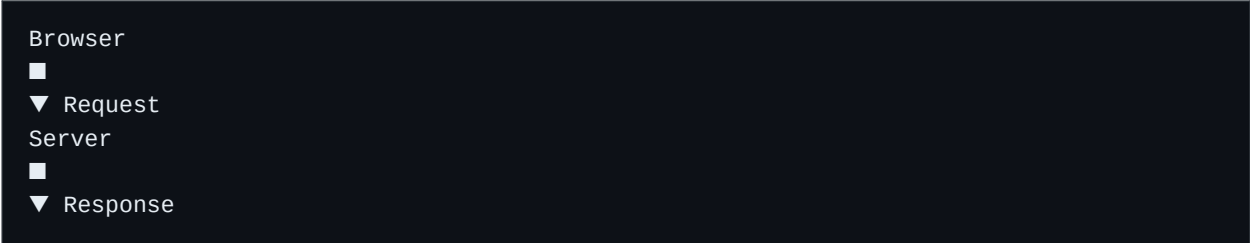
7. HTTP Status Codes — A Pentester's View

Code	Meaning	Pentester's Interpretation
200	OK	Request succeeded
201	Created	Usually follows a successful POST
204	No Content	Success, nothing returned — common after DELETE
301/302	Redirect	May reveal login flows or moved/legacy resources
400	Bad Request	May indicate input validation weaknesses
401	Unauthorized	No valid credentials supplied
403	Forbidden	Resource exists but access is denied — authorization boundary
404	Not Found	Real missing resource, OR deliberately hidden to obscure existence
500	Internal Server Error	Possible unhandled exception, info leakage, or logic error

■ Pentester Mindset

Don't celebrate every 500 error. Always ask: Did my input trigger it? Can I reproduce it reliably? Does the error response leak a stack trace, file path, or query fragment? An unreproducible 500 is just noise.

8. Using Burp Suite to Observe Traffic



Instead of guessing what an application is doing, Burp Suite's proxy lets you intercept and inspect every detail of the conversation between browser and server.

■ PRACTICAL LAB — First Burp Interception

Field	Detail
Objective	Capture and inspect a login request/response using Burp Suite
Tools	Burp Suite Community/Pro, Firefox with FoxyProxy
Setup	Configure browser proxy to 127.0.0.1:8080; install Burp's CA certificate
Steps	1) Turn Intercept ON in Proxy tab 2) Log into the target app 3) Observe the captured POST /login request 4) Forward it and check the response 5) Send it to Repeater for replay
Expected Output	A visible request containing credentials and a response containing a session token or JWT

9. Real API Analysis Workflow

Before attacking anything, a professional pentester builds an inventory:

- 1. Capture all requests
 -
 - ▼
- 2. List every endpoint
 -
 - ▼
- 3. Identify all HTTP methods
 -
 - ▼
- 4. Note authentication requirements
 -
 - ▼
- 5. Identify IDs and parameters
 -
 - ▼
- 6. Observe responses
 -
 - ▼
- 7. Only then begin security testing

■ Why this matters

This disciplined approach prevents missing important attack surface. Rushing straight to payloads means you'll likely miss the IDOR in an endpoint you never even catalogued.

10. Practical Lab — Building an API Inventory (Juice Shop)

Field	Detail
Objective	Build a complete request/response inventory of a target web app
Tools	OWASP Juice Shop (target), Burp Suite, browser
Actions to capture	Login, View Products, View Profile, Add Item to Basket, Logout

Deliverable Table

Method	Endpoint	Auth Required?	Status
POST	/login	No	200
GET	/profile	Yes	200
GET	/products	No	200
POST	/basket	Yes	201

This is exactly how professional testers document API behavior before writing a single exploit.

11. Mentor Challenge #1 — Privilege Escalation via PATCH

```
PATCH /api/users/15
Authorization: Bearer eyJ...
```

```
{
  "role": "admin"
}
```

Answer the following in point form as an attacker:

- **Why is PATCH more interesting than GET here?** — PATCH modifies state; if authorization is weak, an attacker can change privileged fields instead of just reading data.
- **Which security control should prevent this?** — Server-side field-level authorization / an allow-list that rejects role from being set by non-admin users (broken object property level authorization).
- **Which OWASP API risk applies?** — API3:2023 Broken Object Property Level Authorization (covers Mass Assignment and Excessive Data Exposure).
- **What to observe before testing?** — Whether the JWT's role claim matches the account's real privilege, whether the same request succeeds from a low-privilege token, and whether the API returns 200 vs 403 for the attempted change.

12. API Discovery & Endpoint Mapping Methodology

A senior pentester spends 50–70% of an engagement understanding the application before attempting exploitation. Given only a root domain, the first questions are:

- What APIs exist?
- Which require authentication?
- Which expose sensitive functionality?
- Which endpoints should be tested first?

Think Like a Detective

Given `https://shop.company.com`, don't ask *"Where is the vulnerability?"* — ask *"How does this application work?"* Professionals understand business logic before attacking it.

Building the Endpoint Map

Browsing normally (homepage → login → products → cart → checkout → logout) while Burp captures traffic produces an inventory automatically:

```
GET /api/products
POST /api/login
GET /api/profile
POST /api/cart
DELETE /api/cart/3
POST /api/logout
```

13. Categorizing Endpoints & Authentication Boundaries

Group by Function

Category	Example Endpoints
Authentication	POST /login, POST /logout, POST /register, POST /refresh-token
User	GET /profile, PATCH /profile, GET /orders
Admin	GET /admin/users, DELETE /admin/user/12, POST /admin/create-user
Products	GET /products, GET /products/5, POST /reviews

Map Authentication Boundaries

Boundary	Example Endpoints
Public (no login)	GET /products, GET /categories, POST /login
Protected (auth required)	GET /profile, POST /checkout, GET /orders
Admin-only	DELETE /users, GET /admin/users, PATCH /users/role

■ Attacker Mindset

Admin-only endpoints become high-priority targets for later authorization testing — the question is always: does the server actually enforce this boundary, or only the UI?

14. Identifying User-Controlled Input

Every parameter — in the URL, query string, or JSON body — is a potential attack surface.

```
GET /api/products?id=5 → user controls: id
```

```
POST body:
```

```
{  
  "name": "Ali",  
  "email": "ali@example.com"  
} → user controls: name, email
```

- Write down every parameter discovered, including hidden/optional ones seen only in JavaScript or API docs.
- Numeric, sequential IDs (id=5, /orders/125) are prime IDOR candidates.

15. JavaScript Recon for Hidden Endpoints

Many APIs are never exposed through visible UI links, but JavaScript bundles reveal them through fetch()/axios calls.

```
fetch("/api/orders")
fetch("/api/profile")
fetch("/api/admin/users")
```

Three endpoints discovered here without clicking anything in the browser.

■ PRACTICAL LAB — Extracting Endpoints from JS Bundles

Field	Detail
Objective	Discover hidden/undocumented endpoints from client-side JavaScript
Tools	Burp Suite (Engagement Tools → Search), LinkFinder, JSFScan, grep
Command	python3 linkfinder.py -i https://target/main.js -o cli
Alternative	grep -Eo "(/api/[a-zA-Z0-9_-]+)" main.js sort -u
Expected Output	A list of endpoint paths referenced in the bundle, some never linked in the UI

■ Defensive View

Do not rely on hiding an endpoint from the UI as a security control ("security through obscurity"). If the JS or mobile app can call it, an attacker can find and call it too — enforce real authorization server-side.

16. Building a Recon Table & Prioritizing Targets

Endpoint	Method	Auth	Purpose	Notes
/login	POST	No	Login	Returns JWT
/products	GET	No	Product list	Public
/profile	GET	Yes	User profile	Uses Bearer Token
/orders	GET	Yes	User orders	Returns IDs
/admin/users	GET	Admin	User management	High priority

Priority Tiers

Priority	Functionality
High	Authentication, Authorization, Admin APIs, File Upload, Payment APIs, Password Reset, User Management
Lower	Blog, Contact, About, FAQ

Rationale: compromising /about has little security impact, while compromising /admin/users can mean full account takeover across the platform.

17. The Full Reconnaissance Workflow

```
Scope
■
▼
Subdomain Enumeration
■
▼
Identify Live Hosts
■
▼
Spider / Crawl
■
▼
Directory Fuzzing
■
▼
JavaScript Analysis
■
▼
Capture Traffic (Burp)
■
▼
List Endpoints
■
▼
Identify Authentication
■
▼
Map Business Logic
■
▼
Only Now... Begin Security Testing
```

■ PRACTICAL LAB — End-to-End Recon Pass

Step	Tool	Example Command
Subdomain enum	subfinder / amass	subfinder -d company.com -o subs.txt
Live host check	httpx	httpx -l subs.txt -o live.txt
Crawl	katana / gospider	katana -u https://shop.company.com -o crawl.txt
Directory fuzz	ffuf	ffuf -u https://target/FUZZ -w wordlist.txt
JS analysis	LinkFinder	python3 linkfinder.py -i live.txt -o results.html
Traffic capture	Burp Suite	Proxy through 127.0.0.1:8080 while browsing

18. Mentor Challenge #2 — Prioritizing a Target List

```
POST /login
GET /profile
PATCH /profile
GET /users/15
GET /users/16
DELETE /users/16
POST /upload
POST /password/reset
GET /admin/users
```

Suggested Answer

Priority	Endpoint	Likely Issue to Investigate
1	GET /users/15 & /users/16	Authorization (IDOR) — can one user access another's data by changing the ID?
2	GET /admin/users	Authorization — is admin access enforced server-side, not just hidden in the UI?
3	POST /upload	File handling — file type/content validation, storage location, execution risk

Runner-ups worth noting: POST /password/reset (account takeover via weak reset logic) and DELETE /users/16 (authorization + business logic).

19. From Recon to Vulnerability Testing

A professional pentest is **not**:

```
Recon → Exploit ■
```

It is:

```
Recon → Analysis → Hypothesis → Verification → Finding → Report ■
```

Building a Testing Plan

Given discovered subdomains and endpoints, ask: *"What is the highest-risk functionality?"*

```
Authentication
■
▼
Authorization
■
▼
Admin Features
■
▼
File Upload
■
▼
Business Logic
■
▼
Input Validation
```

Questions Before Testing (example: PATCH /profile)

- Who can access it?
- What fields can be changed?
- Is authentication required?
- Can users modify only their own profile?
- Does it validate input?

20. Building Hypotheses

Observation:
GET /users/15

Hypothesis:
"Maybe the application only checks if I'm logged in,
not whether I own user 15."

→ This gives a focused, testable hypothesis instead of random guessing.

Worked Example

Observation → GET /orders/125 uses a numeric ID
Question → Can I change it?
Hypothesis → Maybe another user's order is accessible
Verification → Change only the ID (e.g. to /orders/126)
Result → Document the behavior (200 = confirmed IDOR, 403/404 = protected)

■ PRACTICAL LAB — IDOR Hypothesis Testing

Field	Detail
Objective	Verify whether an authenticated user can access another user's object by ID
Tools	Burp Suite Repeater / Intruder
Steps	1) Log in as User A, capture GET /orders/125 2) Send to Repeater 3) Change ID to another user's order 4) Compare status code and response body
Expected (vulnerable)	HTTP 200 with User B's order data returned to User A's session
Expected (secure)	HTTP 403/404 — server checks object ownership, not just authentication

21. Documentation Standards for Findings

Item	Example
Endpoint	/api/profile
Method	PATCH
Authentication	Bearer JWT
Observation	User can update email field
Expected behavior	Only own account should be editable
Actual behavior	(filled in after verification)
Severity	(assigned per CVSS/impact after verification)

■ Discipline Reminder

Don't rely on memory. Document continuously during testing — screenshots, raw requests/responses, and reproduction steps are what turn an observation into a credible finding.

22. Complete Professional Pentest Workflow

```
graph TD; A[Read Scope] --> B[Passive Recon]; B --> C[Active Recon]; C --> D[Subdomain Enumeration]; D --> E[Host Discovery]; E --> F[Crawling]; F --> G[Directory Fuzzing]; G --> H[JavaScript Analysis]; H --> I[Capture API Traffic]; I --> J[Build Endpoint Inventory]; J --> K[Understand Authentication]; K --> L[Understand Authorization]; L --> M[Identify Attack Surface]; M --> N[Create Testing Plan]; N --> O[Begin Controlled Security Testing]; O --> P[Document Findings]; P --> Q[Write Report];
```

Read Scope

Passive Recon

Active Recon

Subdomain Enumeration

Host Discovery

Crawling

Directory Fuzzing

JavaScript Analysis

Capture API Traffic

Build Endpoint Inventory

Understand Authentication

Understand Authorization

Identify Attack Surface

Create Testing Plan

Begin Controlled Security Testing

Document Findings

Write Report

23. Mentor Challenge #3 — Final Recon Exercise

Using OWASP Juice Shop, build an inventory table:

Endpoint	Method	Auth	Purpose	Priority
/login	POST	No	Authentication	High
/profile	GET	Yes	User data	High
/basket	POST	Yes	Shopping cart	Medium
/products	GET	No	Product listing	Low

Reflection Questions

- Which endpoints deserve your attention first, and why (impact vs. exposure)?
- What security assumptions does the application appear to make (e.g. trusting client-side role, trusting sequential IDs, trusting the JWT payload without server-side re-validation)?

24. OWASP API Security Top 10 — Quick Reference

ID	Risk	One-Line Description
API1	Broken Object Level Authorization	Users access objects (IDs) they don't own — classic IDOR
API2	Broken Authentication	Weak login, token, or session handling
API3	Broken Object Property Level Authorization	Mass assignment / excessive data exposure of object fields
API4	Unrestricted Resource Consumption	No rate limiting / resource exhaustion (DoS)
API5	Broken Function Level Authorization	Low-privilege users reach admin-only functions
API6	Unrestricted Access to Sensitive Business Flows	Automation abuse of legitimate flows (e.g. bulk purchasing, scalping)
API7	Server Side Request Forgery (SSRF)	API fetches a URL supplied by the client
API8	Security Misconfiguration	Verbose errors, default configs, missing headers
API9	Improper Inventory Management	Undocumented/old API versions still reachable
API10	Unsafe Consumption of APIs	Blind trust of data from third-party/internal APIs

Reference: OWASP API Security Project — verify current numbering/wording against the latest official release before citing in a report.

25. Suggested Learning Roadmap

Stage	Topics
1. Authentication Testing	Login flows, password reset, session management, JWT analysis
2. Authorization Testing	Horizontal access control, vertical access control, object-level authorization
3. Input Validation	SQL Injection, NoSQL Injection, Command Injection, SSTI
4. Business Logic Testing	Coupon abuse, payment manipulation, race conditions, workflow bypass
5. File Upload Testing	File type validation, content validation, storage behavior
6. API Security (OWASP Top 10)	API1–API10, real-world methodology, Burp Suite practical labs

■ Senior Pentester Advice

Beginners think: "I know SQLi, XSS, SSRF, IDOR." Professionals think: "I understand this application." Deep understanding of how an app is built naturally reveals where to apply specific testing techniques — technique knowledge without application understanding leads to random, low-value testing.

Recommended Next Course

API Pentesting Masterclass (Beginner → Advanced → Bug Bounty) — 20 lessons combining attacker mindset, Burp Suite practice, OWASP API Security Top 10, Juice Shop & crAPI labs, FastAPI-based vulnerable examples, real bug bounty case studies, and hands-on exercises after every lesson.

These notes are for authorized security testing, personal lab environments, and CTF/training platforms (e.g. OWASP Juice Shop, crAPI) only. Always operate within a defined scope and written authorization.